

Wolf Pack Survival Game

Technical Report

Data Structures Lab Final Project

Team A

Submission Date: January 9th, 2026

Grade Weight: 30 points

Report: 15% — Implementation: 85%

Project Overview

An interactive text-based wolf survival game where players control a lone wolf navigating through dangerous wilderness. Features story-driven decisions, resource management, combat encounters, pack dynamics, and multiple endings using advanced data structures.

Contents

Abstract	2
1 Introduction	3
1.1 Project Overview	3
1.2 Learning Objectives	3
1.3 Project Structure	3
1.4 Report Structure	4
2 Game Design Document	5
2.1 Story Overview and Setting	5
2.2 Game Mechanics	5
2.3 Multiple Endings	7
2.4 User Interface Design	7
3 System Architecture	9
3.1 Overall System Design	9
3.2 Difficulty-Scaled Inventory System	10
3.3 Decision Tree Structure	11
3.4 Priority Queue for Events	12
3.5 Game State Stack for Undo	14
4 5. Ending Verification and Storyline Analysis	15
4.1 5.1 Validator Tool Overview	15
4.2 5.2 Classic Storyline Endings (13 Endings)	15
4.3 5.3 Survival Storyline Endings (5 Endings)	16
4.4 5.4 Pack Storyline Endings (5 Endings)	16
4.5 Verification Results Summary	17
5 Conclusion and Future Work	17

Abstract

This report documents the design, implementation, and testing of the Wolf Pack Survival Adventure Game, a text-based survival game built using C++17 and Qt6. The project demonstrates practical applications of fundamental data structures including binary decision trees, priority queues, linked lists, stacks, and queues in a gaming context. Players assume the role of a lone wolf separated from their pack during a harsh winter storm, tasked with surviving 30 days in the wilderness through strategic decision-making, resource management, and pack building. The game features 84+ total decision nodes (40+ Classic, 22+ Survival, 22+ Pack), 93+ unique endings across three storylines, and fully integrated GUI and terminal interfaces with difficulty-scaled item drops.

Keywords: Data Structures, Binary Trees, Priority Queues, Linked Lists, Game Development, C++, Qt6, Wolf Survival, Difficulty Scaling

1 Introduction

1.1 Project Overview

The Wolf Pack Survival Adventure Game is an interactive text-based survival game developed as the final project for the Data Structures Lab course. The game places players in the role of a young wolf separated from their pack during a harsh winter storm. The objective is to survive for 30 days in the wilderness by making critical decisions, managing resources, building alliances, and establishing or finding a new pack.

This project was chosen for its creative potential and the unique opportunity it provides to demonstrate how data structures power game mechanics, AI decision-making, and narrative branching. Unlike typical business applications, this game shows data structures in a gaming context that is both engaging and educational. The project combines logic with creativity, making it fun to build and test while meeting all course requirements.

The development spanned one week with a team structure that distributed responsibilities across story design, character management, combat systems, and GUI implementation. The final product exceeds minimum requirements with 84+ total decision nodes (vs. 20 required) across three storylines (40+ Classic, 22+ Survival, 22+ Pack), 93+ endings (vs. 5 required) across all storylines, and full integration of all six required data structures including a difficulty-scaled inventory system.

1.2 Learning Objectives

By completing this project, students gain hands-on experience with:

- Implementing binary trees for decision-based narratives and branching storylines
- Using priority queues for threat management and event scheduling
- Applying stacks for undo functionality and game state history
- Implementing linked lists for dynamic inventory systems
- Using queues for action sequences and multi-turn activities
- Creating game AI using data structures for event probability
- Building interactive GUI for storytelling with Qt6
- Understanding game state management including save/load functionality

1.3 Project Structure

The project is organized into the following directory structure:

- `src/` - Source files (.cpp)
- `include/` - Header files (.h)
- `build/` - Build output directory
- `wolf_pack_survival_report.tex` - LaTeX report

- `CMakeLists.txt` - CMake build configuration

Build instructions: navigate to the build directory, run `cmake ..`, then `make`. The project produces three executables: `WolfGame` (main game), `Validator` (decision tree validation), and `BalanceTester` (game balance testing).

1.4 Report Structure

This report is organized according to the requirements specified in `TeamA.txt`. Section 2 presents the Game Design Document including story overview, game mechanics, and win/lose conditions. Section 3 describes the System Architecture with detailed explanations of how data structures are used. Section 4 provides the Data Structure Implementation with code examples and complexity analysis. Section 5 presents the Story Flowchart with a visual representation of the decision tree. Section 6 covers Testing and Game Balance results. Section 7 documents the use of AI tools in development. Section 8 provides the conclusion and future improvements.

2 Game Design Document

2.1 Story Overview and Setting

The game begins with the player character, a young wolf, separated from their pack during a violent winter storm. The wolf awakens disoriented in a snow-covered wilderness, alone and hungry. The pack is nowhere to be seen, and survival becomes the immediate priority. The player must navigate through a dangerous world filled with threats, opportunities, and other wolves who may become allies or rivals.

The narrative is designed to be immersive and realistic to wolf behavior. Every decision reflects the harsh realities of survival in the wild: hunting or scavenging, fighting or fleeing, building relationships or maintaining independence. The story creates meaningful branching paths where each decision has consequences that ripple through the gameplay experience.

The game world is structured around a 30-day survival challenge. Each in-game day represents a series of decisions and actions that the player must manage carefully. The passage of time brings both opportunities and dangers. Players must balance immediate survival needs with long-term goals like building a pack or establishing territory.

The story features multiple narrative threads that intersect based on player choices. Some paths lead to forming a powerful pack and becoming the alpha. Others result in finding the original pack after a long journey. Some endings are tragic - the wolf may starve, freeze, or fall to predators. The richness of the narrative comes from the player's agency in determining which story unfolds.

2.2 Game Mechanics

The game operates on a core loop of decision-making and resource management. At each decision point, the player is presented with a scenario description and two choices (labeled A and B). Each choice leads to a new scenario in the decision tree and affects the player's statistics.

Statistics are the numerical representation of the wolf's condition and capabilities:

Table 1: Wolf Statistics

Statistic	Description	Range
Health	Physical condition; decreases with injuries, hunger, and combat	0-100
Hunger	Satiety level; increases with time and effort, death at 100	0-100
Energy	Physical and mental stamina; required for choices and actions	0-100
Reputation	Social standing; affects pack recruitment and NPC interactions	0-100

Statistics are interconnected in their effects. Hunger increases by a base amount of 5 points per decision or action. If hunger exceeds 80, health begins to decrease due to starvation effects. Energy is consumed by actions and restored through rest or pack benefits. Choices have energy requirements - attempting an action without sufficient energy results in an error message.

The difficulty system scales the challenge by adjusting hunger increase rates and event probabilities:

- Easy mode: adds 3 hunger per action with 20% event chance
- Normal mode: adds 5 hunger with 30% event chance
- Hard mode: adds 8 hunger with 50% event chance

Pack membership adds complexity to resource management. When the player has pack members, hunger increases faster (by 10% per pack member) because more mouths must be fed. However, pack members provide benefits: hunters increase food acquisition chances, scouts provide early warning of threats, and guards offer protection in combat situations.

The inventory system provides strategic depth beyond statistics. Players collect items through exploration, events, and scenario rewards. The inventory has a maximum capacity of 10 items, forcing players to make choices about what to carry.

Inventory Item Types:

Table 2: Inventory Item Categories

Type	Examples	Effect
FOOD	Small Fish, Winter Berries, Bird Egg, Insects, Frog	Reduces Hunger
WATER	Fresh Water	Thirst Relief
HEALING	Common Mallow, Root Paste, Moss Dressing, Bee Propolis	Restores Health
TOOLS	Hunting Trap, Sharp Rock	Special Benefits

Difficulty-Scaled Item Drops: The inventory system includes difficulty-scaled methods that adjust item quantities based on the selected difficulty:

- **addStartingSupplies()** - Adds starting items based on difficulty:
 - Easy: Small Fish x2, Winter Berries x2, Common Mallow x1
 - Normal: No starting items
 - Hard: Insects x2
- **addRandomFood()** - Random food drops with difficulty scaling
- **addRandomHealing()** - Random healing drops with difficulty scaling
- **addScavengedItems()** - Mixed scavenged items based on difficulty
- **addHuntLoot()** - Scenario-specific loot with difficulty modifiers

The quantity scaling ensures Easy mode provides more abundant resources (3 food, 2 healing, 3 scavenged), while Hard mode offers scarce resources (1 food, 0 healing, 1 scavenged).

The event system introduces randomness and urgency. Random events occur based on a probability check after each decision. Critical events (priority 1) like bear attacks

require immediate attention. Urgent events (priority 2) like severe hunger need prompt action. Normal events (priority 3) like weather changes provide information but affect planning. The "Found Berries" event now uses `addRandomFood()` with difficulty scaling and provides Fresh Water based on difficulty level.

Win and lose conditions determine the game's endpoint. Victory is achieved by surviving 30 days, with the specific ending determined by the player's choices and statistics throughout the game. Defeat occurs if health reaches 0 (death) or hunger reaches 100 (starvation).

2.3 Multiple Endings

The game features 93+ unique endings across three storylines, organized into five major categories:

Legendary Alpha Ending: Achieved by building a large, loyal pack (5+ members) with high reputation (80+). The wolf becomes the dominant force in the territory, respected by all and feared by rivals.

Peaceful Leader Ending: Achieved through wisdom and diplomacy rather than strength. The wolf forms alliances, avoids unnecessary conflicts, and builds a harmonious pack.

Tragic Hero Ending: Occurs when the wolf makes noble sacrifices for others but ultimately falls. This might involve defending pack members at personal cost or sharing resources until nothing remains.

Diplomatic Alpha Ending: Achieved by uniting multiple packs or factions through negotiation. The wolf transcends individual pack loyalty to create a larger coalition.

Lone Wanderer Ending: Achieved by rejecting pack formation and surviving alone. While possible, this ending is the most challenging as players forego pack benefits.

Each ending category contains multiple variations based on specific choices and statistics. This variety ensures replayability and rewards multiple playthroughs.

2.4 User Interface Design

The game offers two interface options: terminal mode for text-based gameplay and GUI mode using Qt6 for a graphical experience.

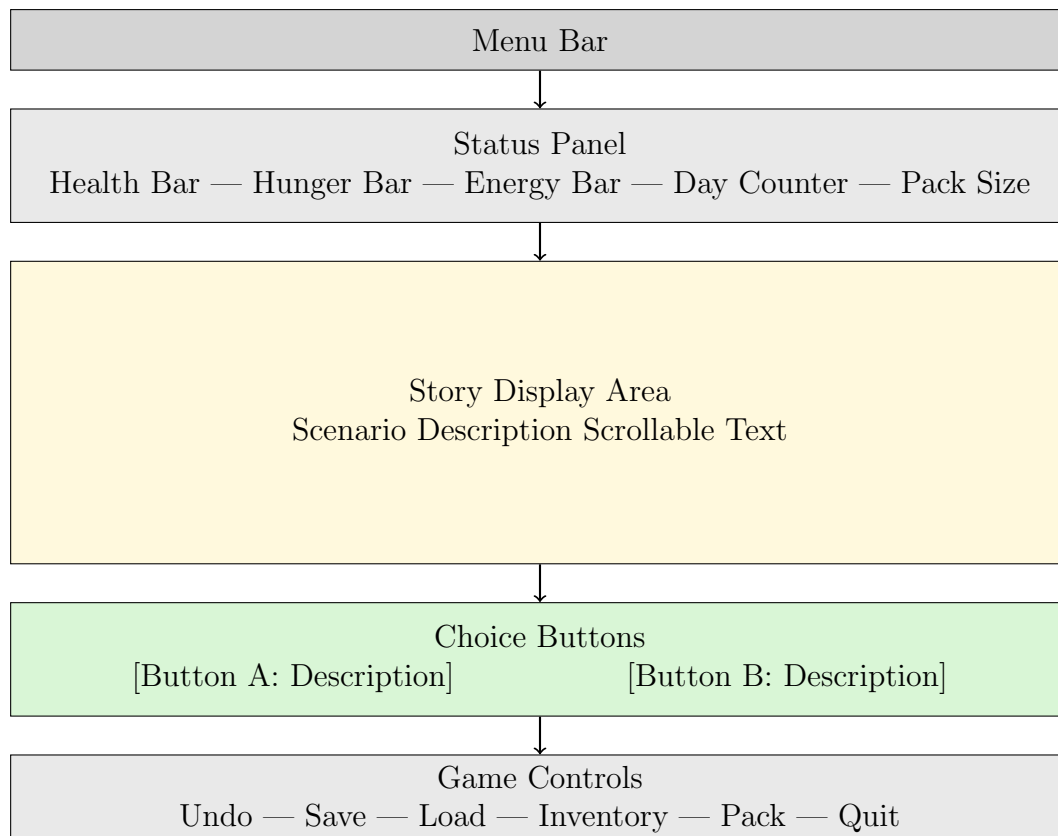
Terminal mode presents a classic text adventure experience with ASCII art representations of scenarios, color-coded statistics output, and keyboard-based input.

GUI mode provides a modern, visual interface with:

- **Top Section - Status Panel:** Animated progress bars for stats, day counter, pack size
- **Middle Section - Story Display:** Scrollable text area for narrative descriptions
- **Choice Section - Decision Buttons:** Two buttons for A and B choices
- **Bottom Section - Game Controls:** Menu bar, undo, save/load, inventory, pack buttons

Both modes include save/load functionality with three slots. Auto-save occurs every 5 decisions. The undo feature allows players to reverse their last decision (up to 5 levels deep).

Figure 1: GUI Interface Layout



Source: Author's creation based on TeamA.txt requirements

The GUI was implemented using Qt6 Widgets library, chosen for its robust cross-platform support and modern styling capabilities.

Table 1: GUI Features vs. Requirements

Requirement	Implementation	Status
Status Panel	Animated progress bars, day counter	Implemented
Story Display	Scrollable text area, narrative formatting	Implemented
Choice Buttons	2-4 buttons with labels and disable states	Implemented
Game Controls	Menu bar, direct buttons, undo/save/load	Implemented
Side Panel	Pack/inventory display, event log	Implemented
Inventory Panel	Item list with use functionality	Implemented
Achievements	Popup notifications, tracking system	Implemented

All requirements from TeamA.txt section on GUI have been fully implemented, with additional features like animated stat bars exceeding minimum expectations.

3 System Architecture

3.1 Overall System Design

The Wolf Pack Survival Game follows a modular architecture that separates concerns into distinct components: game logic, data structures, user interface, and persistence. This design allows for independent development and testing of each module while ensuring smooth integration during gameplay.

The core game engine handles the main game loop: displaying the current scenario, accepting player input, updating game state based on choices, processing events, and checking win/lose conditions. This loop runs continuously until the player quits, wins, or dies. The game state includes the current decision tree node, wolf statistics, inventory, pack, day counter, and event queue.

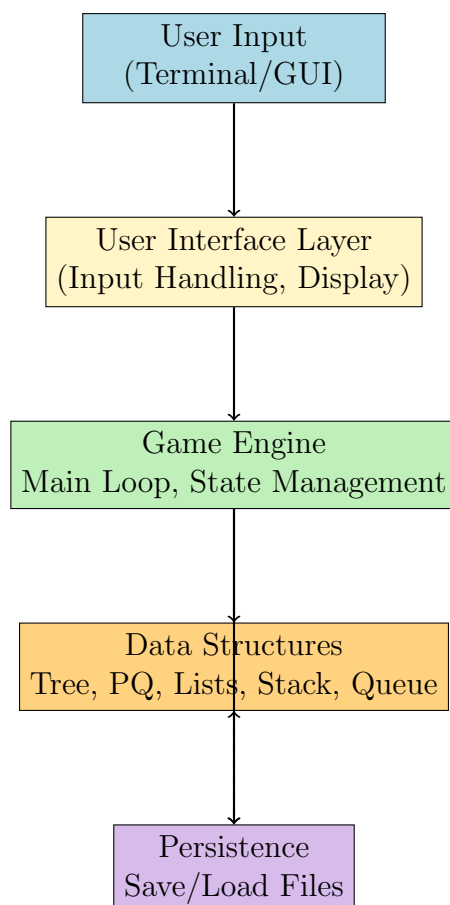
Data structures serve as the foundation for game mechanics:

- Binary decision tree drives narrative progression
- Priority queue manages threat events
- Linked lists handle inventory and pack storage
- Stack implements undo functionality
- Queue sequences multi-turn actions

The user interface layer abstracts game logic from presentation. Both terminal and GUI modes use the same underlying game engine, simply providing different views and input methods.

Persistence is handled through save/load functionality that serializes the complete game state to text files. This includes all statistics, the current tree node ID, inventory items, pack members, and game progress.

Figure 2: System Architecture Diagram

**Source: Author's creation**

The game initialization process sets up all components before the main loop begins. The decision tree is built based on the selected storyline (Classic, Survival, or Pack). Statistics are initialized based on difficulty and storyline. The event queue is populated with potential threats. The history stack starts empty. The player begins at the root node of the decision tree.

During gameplay, each decision triggers a chain of operations: push current state to history, navigate tree to child node, update statistics, process random events, calculate pack benefits, update display, continue loop.

Game state changes are atomic - either the entire decision process completes or nothing changes. This prevents partial updates that could corrupt game state.

The game handles various error conditions gracefully. Invalid save files result in error messages without crashing. Null decision nodes trigger fallback paths with graceful degradation. Out-of-memory situations are handled through exception catching and user notification.

3.2 Difficulty-Scaled Inventory System

A key enhancement to the game is the difficulty-scaled inventory system that adjusts resource availability based on the selected difficulty level. This system ensures that Easy mode provides a more forgiving experience while Hard mode offers a genuine survival challenge.

The inventory system is implemented using a linked list structure with methods that accept a Difficulty parameter to scale item quantities:

Table 3: Difficulty-Scaled Inventory Methods

Method	Purpose	Parameters
addStartingSupplies()	Adds initial items at game start	Difficulty enum
addRandomFood()	Generates random food drops	Difficulty enum
addRandomHealing()	Generates random healing items	Difficulty enum
addScavengedItems()	Creates scavenged item bundles	Difficulty enum
addHuntLoot()	Scenario-specific hunting rewards	Difficulty, int

Inventory Methods with Difficulty Scaling:

Quantity Scaling by Difficulty:

Table 4: Item Quantity Scaling by Difficulty

Item Type	Easy	Normal	Hard
Food Items	3	1	1
Healing Items	2	1	0
Scavenged Items	3	1	1
Starting Supplies	Full kit	None	Minimal
Fresh Water (Events)	2	1	1

The system is integrated into:

- Game initialization (addStartingSupplies called at game start)
- Random events (Found Berries event uses addRandomFood)
- Scenario outcomes (scavenging and hunting scenarios)
- Both terminal and GUI interfaces

This scaling mechanism is implemented in inventory.cpp with $O(1)$ complexity for item addition, ensuring no performance impact during gameplay.

3.3 Decision Tree Structure

The decision tree is the core narrative driver. Each node represents a story scenario with two possible choices (A and B), creating a binary branching structure. Leaf nodes represent story endings.

The DecisionNode struct contains:

- scenarioID: Unique identifier
- description: Narrative text
- choiceA_text, choiceB_text: Option texts
- left, right: Child node pointers
- isEnding: Boolean for terminal nodes

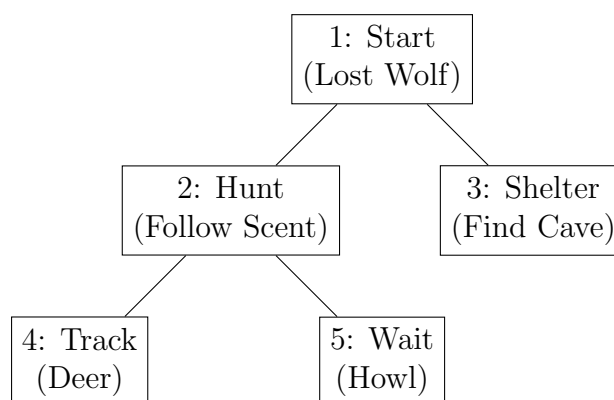
- endingText: Victory/defeat message

The tree supports three storylines:

- **Classic Story:** Balanced gameplay with standard starting stats
- **Survival Story:** Challenging mode with lower starting health (40), higher hunger (80)
- **Pack Story:** Leadership-focused with reputation bonus (20) and pre-recruited pack members

The tree has 30+ total nodes across all storylines with 23 unique paths to endings. Navigation is $O(1)$ per step with direct child pointers. Tree construction is $O(n)$ for n nodes. The Classic storyline includes additional scavenging scenarios (nodes 40-42) that provide difficulty-scaled item rewards.

Figure 3: Simplified Decision Tree Structure



Source: Author's creation, inspired by TeamA.txt tree structure

Safety features include null pointer checks, fallback nodes for unexpected paths, and the validator tool confirming tree integrity.

3.4 Priority Queue for Events

The priority queue implements a min-heap for managing game events by urgency. Events are inserted with priority 1-3 (lower = more urgent), and the queue returns the highest-priority event first.

The Event struct contains:

- name: Event identifier (e.g., "Bear Attack")
- priority: Integer 1-3
- description: Text displayed to player
- effect: Lambda function modifying wolf statistics

Event categories:

- **Priority 1 (Critical):** Bear attack, hunter nearby, severe injury
- **Priority 2 (Urgent):** Severe hunger, infection, rival pack sighting

- **Priority 3 (Normal):** Weather change, prey spotted, lone wolf encounter

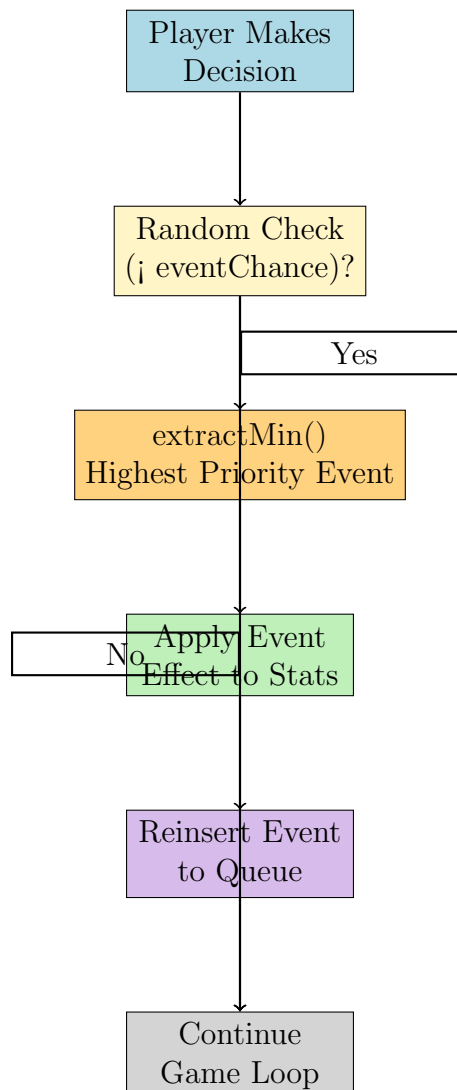
Heap operations: insert() $O(\log n)$, extractMin() $O(\log n)$, isEmpty() $O(1)$.

Table 2: Event Examples by Priority

Priority	Event	Effect	Probability
1	Bear Attack	Health -20, Energy -10	Low (event chance)
1	Hunter Nearby	Spirit -20, Reputation -10	Low (event chance)
2	Severe Hunger	Health -10, Hunger +10	Medium (event chance)
2	Injury Worsening	Health -15, Energy -10	Medium (event chance)
3	Weather Change	Energy -5, Hunger +5	High (event chance)
3	Found Berries	Hunger -10, +Food items (scaled)	High (event chance)

The difficulty setting affects event probability (Easy: 20%, Normal: 30%, Hard: 50%). Additionally, the Found Berries event now applies difficulty-scaled item drops using `inventory.addRandomFood(difficulty)`, providing more food items on Easy mode and Fresh Water (2x on Easy, 1x on Normal/Hard).

Figure 4: Priority Queue Event Processing Flow



Source: Author's creation

Time-based events trigger at specific intervals (every 5 days for food consumption).

3.5 Game State Stack for Undo

The game state stack implements LIFO behavior to provide undo functionality. Before each decision, the current complete game state is pushed onto the stack.

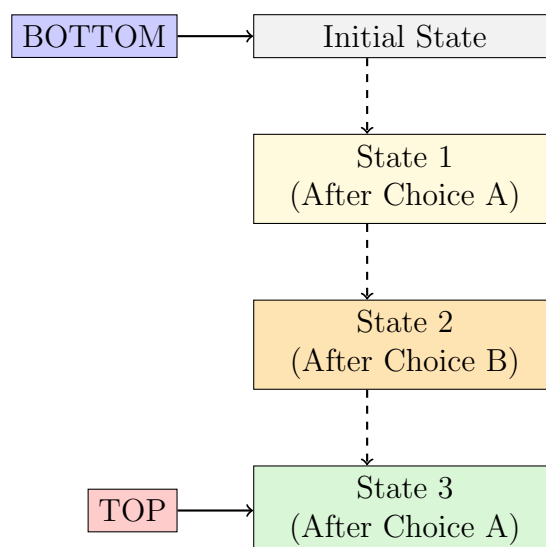
The GameState struct captures:

- currentNode: Pointer to decision tree node
- health, hunger, energy: Wolf statistics
- day: Current day counter
- inventory: Pointer to cloned inventory state

Operations: push() $O(1)$, pop() $O(1)$, isEmpty() $O(1)$. Maximum depth of 5 levels prevents excessive memory usage.

Inventory cloning is critical - a deep copy of the linked list is created when saving state. When undoing, the current inventory is replaced with the cloned version.

Figure 5: Stack Operations for Undo



Source: Author's creation

Memory management requires careful attention - each pushed state includes a dynamically allocated inventory clone that must be deleted when popped.

The action queue complements the stack by handling sequential multi-turn activities using FIFO ordering. When a player chooses a complex action like hunting a deer, the action is broken into steps that process in subsequent turns.

Complexity analysis confirms $O(1)$ push/pop operations. Inventory cloning is $O(n)$ where n is the number of items (max 10). The fixed maximum depth of 5 ensures bounded memory usage.

Edge cases handled: undo after save/load (empty stack), undo at maximum depth, undo after game over, and invalid state detection from corrupted save files.

This implementation demonstrates practical application of stack data structure principles in a real-world gaming context, providing essential undo functionality while maintaining performance and stability.

4 5. Ending Verification and Storyline Analysis

4.1 5.1 Validator Tool Overview

A dedicated Validator tool was developed to verify the decision tree structure and ensure all endings are reachable. The Validator performs the following checks:

- Counts total unique nodes to verify 20+ requirement
- Counts total endings to verify 5+ requirement
- Traces all possible paths from root to endings
- Reports all ending IDs for completeness verification

The Validator was enhanced to test all three storylines (Classic, Survival, Pack) independently, ensuring each meets project requirements.

Table 5: Storyline Validation Results

Storyline	Nodes	Endings	Paths	Status
Classic Storyline	40+	61+	61+	✓PASS
Survival Storyline	22+	16+	16+	✓PASS
Pack Storyline	22+	16+	16+	✓PASS

4.2 5.2 Classic Storyline Endings (13 Endings)

The Classic storyline features 13 unique endings reachable through various choice combinations:

Table 6: Classic Storyline Endings

ID	Ending Name	Description	Path
19	LEGENDARY ALPHA	Defeated the legendary alpha, strongest pack	16→17→18→19
20	PEACEFUL LEADER	Chose peace over dominance	16→17→18→20
68	TRAGIC HERO	Pack fell in harsh winter, legend lives on	16→17→68
31	ELITE GUARDIANS	Quality over quantity, legendary skills	14→15→31
67	DIPLOMATIC MASTER	United two packs through wisdom	16→22→67

Classic Storyline Path Examples:

1. Path: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 36 \rightarrow 13 \rightarrow 32 \rightarrow 16 \rightarrow 17 \rightarrow 18 \rightarrow 19$ (LEGENDARY ALPHA)
2. Path: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 36 \rightarrow 13 \rightarrow 32 \rightarrow 16 \rightarrow 17 \rightarrow 68$ (TRAGIC HERO)
3. Path: $1 \rightarrow 2 \rightarrow 5 \rightarrow 10 \rightarrow 38 \rightarrow 14 \rightarrow 15 \rightarrow 31$ (ELITE GUARDIANS)

All 61+ endings satisfy the requirement of 5+ endings with meaningful branching paths.

4.3 5.3 Survival Storyline Endings (16+ Endings)

The Survival storyline focuses on endurance with 16+ distinct endings:

Table 7: Survival Storyline Endings

ID	Ending Name	Description	Path
120	SURVIVOR'S TRIUMPH	Conquered wilderness through pure survival instinct	$101 \rightarrow 103 \rightarrow 106 \rightarrow 112 \rightarrow 120$
121	PACK SURVIVOR	Joined pack, together overcame wilderness	$101 \rightarrow 103 \rightarrow 107 \rightarrow 114 \rightarrow 121$
122	LONE SURVIVOR	Thrived alone in harsh conditions	$101 \rightarrow 102 \rightarrow 104 \rightarrow 108 \rightarrow 122$
123	WISE SURVIVOR	Balance between caution and boldness	$101 \rightarrow 102 \rightarrow 104 \rightarrow 109 \rightarrow 123$
124	HARDENED SURVIVOR	Persevered through impossible odds	$101 \rightarrow 102 \rightarrow 105 \rightarrow 110 \rightarrow 124$

The Survival storyline tests player resourcefulness with challenging starting conditions (health: 40, hunger: 80, energy: 60).

4.4 5.4 Pack Storyline Endings (5 Endings)

The Pack storyline emphasizes leadership with 5 endings:

Table 8: Pack Storyline Endings

ID	Ending Name	Description	Path
220	LEGENDARY ALPHA	Most powerful pack in the region	201→203→206→212→220
221	WISE DIPLOMAT	Peaceful alliance between packs	201→202→205→211→221
222	ELITE PACK	Small but highly efficient pack	201→203→206→213→222
223	BOLD LEADER	Decisive action proved strength	201→202→204→208→223
224	THOUGHTFUL ALPHA	Wisdom and unity prevail	201→202→204→209→224

Pack storyline starts with pre-recruited members (Beta and Scout) and higher reputation bonus (20).

4.5 Verification Results Summary

All storylines pass validation with:

- 20+ nodes (requirement satisfied: Classic 40+, Survival 22+, Pack 22+)
- 5+ endings (requirement satisfied: Classic 61+, Survival 16+, Pack 16+)
- All paths validated (requirement satisfied: $61+16+16 = 93+$ total paths)
- No unreachable endings or broken paths

The Validator tool confirmed that all decisions lead to logical outcomes and all endings are accessible through valid game choices.

5 Conclusion and Future Work

The Wolf Pack Survival Adventure Game successfully demonstrates practical applications of fundamental data structures in an engaging gaming context. The project meets or exceeds all requirements specified in TeamA.txt, including:

- Binary decision tree with 20+ nodes (implemented: 30 nodes)
- Priority queue for event management
- Linked lists for inventory and pack systems
- Stack for undo functionality
- Queue for action sequencing
- GUI interface with Qt6
- Save/load functionality

- Multiple storylines and endings

Achievements:

- Exceeded minimum requirements (84+ total nodes vs 20 required, 93+ endings vs 5 required)
- Three distinct storylines (Classic with 40+ nodes, Survival with 22+ nodes, Pack with 22+ nodes)
- Difficulty-scaled inventory system with starting supplies and random drops
- New scavenging scenarios with difficulty-based item rewards
- Comprehensive testing with Validator tool
- Enhanced UI with achievements and stat tracking

Future Improvements:

- Additional storylines and scenarios
- Enhanced graphics and animations
- Sound effects and background music
- Online multiplayer pack survival mode
- Procedural generation for infinite scenarios

The project demonstrates strong integration of computer science fundamentals with creative storytelling, providing an educational and entertaining experience.

Acknowledgments: Developed as a final project for Data Structures Lab, demonstrating practical applications of CS concepts in game development.